DRIFT-Repair: Diagnosing and Repairing State Drift in Multi-Turn Retrieval-Augmented LLM Agents

Anonymous Authors

Abstract

Multi-turn retrieval-augmented language-model agents increasingly combine conversation history, retrieval, tools, and memory. Yet their failures are often not isolated hallucinations: an agent may gradually drift away from the user's current goal, active constraints, retrieved evidence, tool observations, or updated memory. We formalize this phenomenon as *state drift*. We introduce a taxonomy covering goal, constraint, evidence, retrieval, tool-state, memory, and abstention drift, and we propose DRIFTBENCH, a diagnostic benchmark with controlled state perturbations and state-level annotations. We further present SAGE-R, a model-agnostic state-graph framework that diagnoses drift and triggers typed repair actions including rollback, re-retrieval, tool cross-validation, clarification, and abstention. The current repository implements a deterministic v0 scaffold with generated examples, baseline proxies, repair traces, analysis tables, and a human-validation packet; these artifacts validate the research pipeline before external datasets, model-backed experiments, and completed human annotation are added.

1 Introduction

Retrieval-augmented generation (RAG) is widely used to ground language models in external evidence, while tool-augmented agents extend LLMs with APIs, database operations, and long-horizon workflows. However, realistic use cases are rarely single-turn. A user often changes goals, adds constraints, corrects earlier assumptions, asks unanswerable questions, or provides partial information. A reliable agent must therefore maintain an evolving state over goals, constraints, evidence, tools, and memory.

Recent benchmarks show that this setting remains difficult. MTRAG evaluates end-to-end human-generated multi-turn RAG conversations and reports that state-of-the-art systems struggle with later turns, unanswerable questions, non-standalone questions, and multi-domain settings [1]. Tool-agent benchmarks such as τ-bench evaluate dynamic user-agent-tool interactions and compare the final database state with the intended goal state, exposing reliability gaps in realistic multi-turn domains [3]. DialogTool further argues that tool-use benchmarks must consider stateful multi-turn interactions rather than only stateless single-turn tool selection [4].

We argue that these failures are better understood as *state drift*: the agent's working state gradually becomes inconsistent with the active task state. Unlike ordinary hallucination, state drift is trajectory-level. It may originate in an early wrong assumption, stale retrieval query, obsolete memory, or contradicted tool observation, and then propagate through later turns.

This paper pursues three questions:

1. How can state drift in multi-turn RAG agents be defined and diagnosed?
2. Can we construct a benchmark with state-level annotations rather than final-answer labels only?
3. Can typed repair actions recover from drift more reliably than generic self-reflection?

Contributions. The intended full paper makes four contributions:

- We formalize state drift in multi-turn retrieval-augmented LLM agents and introduce a seven-part taxonomy.
- We construct DRIFTBENCH, a diagnostic benchmark with controlled perturbations and state-level labels for active goals, constraints, evidence support, tool states, and abstention decisions.
- We propose SAGE-R, a state-graph diagnosis and typed repair framework that is model-agnostic and inference-time.
- We evaluate recovery, state consistency, faithfulness, cost, and over-repair across multi-turn RAG and tool-use settings.

The current v0 package implements the taxonomy, state graph, deterministic DRIFTBENCH scaffold, SAGE-R repair traces, baseline proxies, analysis tables, and annotation workflow needed to make the later empirical stage reproducible.

2 Related Work

Multi-turn RAG evaluation. MTRAG provides 110 human-generated conversations averaging 7.7 turns across four domains, for 842 tasks [1]. Its results suggest that later turns, unanswerable questions, and non-standalone questions are particularly challenging. MTRAG-UN extends this line toward unanswerable, underspecified, non-standalone, and unclear-response cases [2]. Our work differs by introducing state-level drift annotations and repair outcomes.

RAG evaluation and correction. RAGAS proposes reference-free evaluation dimensions for RAG pipelines,

including retrieval quality, faithful use of retrieved context, and generation quality [5]. Corrective RAG (CRAG) improves robustness when retrieved documents are inaccurate by using a retrieval evaluator and corrective retrieval actions [6]. REAP maintains structured sub-tasks and facts for recursive evaluation and adaptive planning in multi-hop RAG [7]. Our work generalizes correction beyond retrieval quality to evolving multi-turn state.

Tool-augmented agents. τ-bench evaluates agents in dynamic conversations with simulated users, tools, domain policies, and database-state outcomes [3]. DialogTool emphasizes the full life cycle of stateful tool use in multi-turn dialogues, including tool creation, awareness, selection, execution, and role-consistent response generation [4]. We use these benchmarks to study tool-state drift and repair.

Long-term memory. LongMemEval evaluates chat assistants on information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention in sustained interactions [8]. Memory update and temporal reasoning are central to our definition of memory drift.

3 Problem Formulation

At turn t, a multi-turn RAG agent observes conversation history $H_t = \{(u_i, a_i)\}_{i<t}$, the current user instruction Q_t, retrieved passages R_t, tool outputs O_t, previous actions $A_{<t}$, and optional memory M_t. The agent produces actions A_t and a response Y_t. We write the complete observable context as $X_t = (H_t, Q_t, R_t, O_t, A_{<t}, M_t)$.

We define the latent active state as:

$$S_t = \{G_t, C_t, F_t, E_t, T_t, M_t, U_t\}, \tag{1}$$

where G_t is the active user goal, C_t active constraints, F_t factual claims, E_t evidence links, T_t tool/database state, M_t memory state, and U_t the uncertainty or abstention status.

In implementation, S_t is represented as a typed state graph $\mathcal{G}_t = (V_t, E_t)$ whose nodes are goals, constraints, facts, evidence items, tool observations, memory items, and uncertainty states. Each node has a status in $\{\text{ACTIVE}, \text{SUPERSEDED}\}$ and provenance fields recording the source turn, evidence, retrieval, or tool output. Active-state resolution is a function

$$\text{active}(\mathcal{G}_t) = \{v \in V_t : \text{status}(v) = \text{ACTIVE}\}, \tag{2}$$

where later goal shifts supersede incompatible earlier goals, later constraint updates supersede obsolete constraints, and memory updates supersede contradicted memory nodes.

State drift. A drift occurs when the generated action or response is inconsistent with at least one active state component:

$$D_t = \mathbb{I}\left[\exists z \in S_t : \text{conflict}(z, A_t, Y_t) > \tau_z\right]. \tag{3}$$

We further assign a typed drift label and severity,

$$\ell_t \in \mathcal{L}, \quad \mathcal{L} = \{\text{G1}, \text{G2}, \text{E1}, \text{E2}, \text{T1}, \text{M1}, \text{U1}, \text{NONE}\}, \tag{4}$$

Code	Drift type	Example failure
G1	Goal drift	Answers old task after user changes goal
G2	Constraint drift	Violates latest budget/date/source constraint
E1	Evidence drift	Claims unsupported or contradicted by evidence
E2	Retrieval drift	Retrieves for stale intent
T1	Tool-state drift	Ignores tool/database observation
M1	Memory drift	Uses obsolete user preference or fact
U1	Abstention drift	Answers when evidence is absent, or over-refuses

Table 1: Planned taxonomy of state drift in multi-turn retrieval-augmented agents.

with $s_t \in \{\text{low}, \text{medium}, \text{high}\}$ derived from the conflicting active nodes and their provenance.

Diagnosis. The diagnosis function outputs a structured record:

$$\mathcal{D}_t = \text{diagnose}(H_t, R_t, O_t, A_t, Y_t) \tag{5}$$

including drift type ℓ_t, triggering turn, conflicting nodes, severity s_t, module-level signals, and suggested repair r_t.

Repair. Given a diagnosis, repair seeks a corrected state and response:

$$(S'_t, Y'_t) = \text{repair}(S_t, \mathcal{D}_t, H_t, R_t, O_t) \tag{6}$$

optimizing task success, state consistency, faithfulness, and cost.

4 State Drift Taxonomy

This taxonomy is designed to separate ordinary hallucination from trajectory-level state failure. For example, if an agent processes a refund after a tool returns *not eligible*, the failure is not merely a factual hallucination; it is a tool-state conflict. If the user updates a constraint from "under $500''to"under800$" and the agent still filters products by the old constraint, this is memory or constraint drift.

5 DriftBench Construction

DRIFTBENCH will be built by transforming existing multi-turn tasks into controlled diagnostic cases. Each case contains the original conversation, retrieved evidence or tool observations, perturbations, active-state labels, and expected repair actions.

Sources. We plan to use MTRAG/MTRAG-UN for multi-turn RAG, τ-bench or DialogTool for stateful tool use, and optionally LongMemEval for sustained memory.

Controlled perturbations. We will inject or select cases with:

- user goal shifts;
- constraint updates;

- conflicting retrieved evidence;
- stale retrieval queries;
- tool outputs that contradict planned actions;
- memory updates that supersede old preferences;
- unanswerable or underspecified user turns.

Annotation schema. Each case will include active goals, active constraints, supported claims, contradicted claims, tool-state labels, memory supersession links, and gold action among answer, re-retrieve, call tool, ask clarification, abstain, or rollback.

6 Method: SAGE-R

SAGE-R has four stages.

1. State graph construction. We convert the conversation trajectory into a typed graph. Nodes represent user goals, constraints, retrieved evidence, tool calls, tool observations, claims, memories, and responses. Edges include `supports`, `contradicts`, `updates`, `supersedes`, `depends-on`, and `requires-clarification`.

2. Active-state resolution. The framework resolves which historical items remain active. Explicit user updates supersede earlier constraints; tool/database outputs override agent assumptions; assumptions remain defeasible; and uncertain states trigger verification or clarification.

3. Drift detection. Detectors score goal consistency, constraint consistency, evidence faithfulness, retrieval-goal alignment, tool-state consistency, temporal consistency, and abstention calibration.

4. Typed repair. Each drift type triggers a repair policy:
- Goal or constraint drift: rollback to last consistent state and re-plan.
- Evidence or retrieval drift: rewrite query, re-retrieve, verify claims, regenerate.
- Tool-state drift: re-check tool output, cross-validate, or revise action.
- Memory drift: update memory priority and mark superseded facts inactive.
- Abstention drift: choose answer, clarify, retrieve more, or abstain.

7 Experimental Design

Baselines. We compare against vanilla multi-turn RAG, standalone query rewriting, RAGAS-style filtering, self-reflection, ReAct, CRAG-style corrective RAG, and planner-RAG or REAP-style baselines.

SAGE-R inference-time diagnosis and repair

1. Build a typed state graph from conversation turns, retrieved evidence, tool observations, memory updates, and the candidate response.
2. Resolve active state by marking superseded goals, constraints, and memories inactive; tool observations override agent assumptions.
3. Run modular detectors for goal/constraint, retrieval, evidence/abstention, tool-state, and memory drift.
4. Aggregate drift signals into a diagnosis record with type, severity, conflicting nodes, confidence, and suggested repair.
5. If no drift is detected, preserve the response. Otherwise apply the highest-priority typed repair: rollback/replan, rewrite/retrieve, verify claims, cross-check tools, update memory priority, or abstain/clarify.
6. Log a repair trace with checked state, actions, stop condition, and cost for downstream analysis.

Figure 1: SAGE-R algorithm box. The current implementation is a deterministic scaffold; future experiments replace rule modules with model-backed extraction, retrieval, verification, and judging.

Metrics. Final-task metrics include task success, exact match/F1 when applicable, database goal match, and judge/human preference. State metrics include goal consistency, constraint consistency, evidence consistency, tool-state consistency, temporal consistency, and abstention calibration. Repair metrics include recovery rate, repair precision, over-repair rate, time-to-recovery, and token/tool-call overhead.

Human validation. Because LLM judges can be unstable, we will manually validate at least 200–300 cases and report judge-human agreement.

8 Current Scaffold Results

This section records the current deterministic v0 scaffold results. The purpose is to validate data flow, diagnosis traces, repair traces, table generation, and analysis scripts before external datasets and model calls are added. These numbers should not be interpreted as final benchmark claims.

DriftBench v0. The current repository contains 300 toy-equivalent draft examples generated from 21 hand-written templates. The set covers the seven drift types in our taxonomy and includes clean controls. We compare SAGE-R with three deterministic baseline proxies: vanilla RAG, query-rewriting RAG, and self-reflection. Table 2 summarizes the current scaffold metrics.

Table 3 reports paired bootstrap comparisons for the deterministic scaffold. These intervals and p-values validate the reporting code path only; they must be recomputed on real model-backed runs before final claims.

System	Exact	F1	Repair	Over-repair	Tokens
SAGE-R	1.0000	1.0000	0.8533	0.0000	42.08
QueryRewrite	0.2900	0.4765	0.2367	0.0000	4.26
SelfReflect	0.4300	0.7680	0.6133	0.0000	39.25
Vanilla	0.1467	0.0000	0.0000	0.0000	0.00

Table 2: Deterministic DriftBench v0 scaffold results. These numbers validate the pipeline and are not final benchmark claims.

Comparison	Metric	Delta	95% CI	p
SAGE-R — query_rewrite_rag	Exact	0.7100	[0.6600, 0.7567]	0.0000
SAGE-R — query_rewrite_rag	Micro-F1	0.5235	[0.4621, 0.5897]	0.0000
SAGE-R — self_reflection	Exact	0.5700	[0.5132, 0.6233]	0.0000
SAGE-R — self_reflection	Micro-F1	0.2320	[0.2032, 0.2612]	0.0000
SAGE-R — vanilla_rag	Exact	0.8533	[0.8133, 0.8933]	0.0000
SAGE-R — vanilla_rag	Micro-F1	1.0000	[1.0000, 1.0000]	0.0000

Table 3: Paired bootstrap comparisons for the deterministic v0 scaffold. These tests validate the reporting pipeline and are not final statistical claims.

Ablations. Table 4 reports deterministic module-removal ablations. These isolate whether the scaffold can attribute failures to the expected detector family. The diagnose-only variant preserves type detection but disables repair, so it has zero repair cost by construction.

Cost, over-repair, and validation workflow. Table 5 summarizes extra token, retrieval, and tool-call cost proxies. Table 6 reports clean-case over-repair on examples whose gold action is answer-as-is. Table 7 records the human-validation workflow scaffold: a 240-case blinded annotation packet with a 48-case double-annotation subset. The current agreement table uses an answer-key proxy only and must be replaced by completed human annotations before submission.

9 Expected Findings

We expect typed repair to outperform generic self-reflection especially on tool-state, constraint, and retrieval drift. We also expect over-repair to be a real failure mode: some correct responses may be unnecessarily revised. Therefore, the repair policy must include confidence thresholds and abstain from repair when diagnosis is uncertain.

10 Limitations

The benchmark may over-represent controlled perturbations compared with naturally occurring failures. LLM-based extraction and judging may introduce bias. Repair can increase latency and cost. Finally, reliable diagnosis does not guarantee safe deployment in high-stakes domains; domain-specific verification remains necessary.

Variant	Exact	F1	Repair	Tokens
Full	1.0000	1.0000	0.8533	42.08
w/o Goal+Constr.	0.6700	0.8374	0.8067	37.58
w/o Retrieval	0.7633	0.9152	0.8533	42.08
w/o Evid.+Abst.	0.7633	0.8359	0.6633	32.58
w/o Tool	0.8067	0.9139	0.6600	33.96
w/o Memory	0.8600	0.9343	0.7133	36.76
Diagnose-only	1.0000	1.0000	0.0000	0.00

Table 4: Deterministic ablations on DriftBench v0. Module-removal variants isolate diagnosis coverage in the scaffold.

System	Repaired	Tokens	Retrievals	Tools
QueryRewrite	71	4.26	0.2367	0.0000
SAGE-R	256	42.08	0.5667	0.1933
SelfReflect	184	39.25	0.0000	0.0000
Vanilla	0	0.00	0.0000	0.0000

Table 5: Cost logging summary for the deterministic v0 scaffold.

11 Conclusion

This draft proposes a research program for diagnosing and repairing state drift in multi-turn retrieval-augmented LLM agents. The intended contribution is not another monolithic agent, but a diagnostic framework, benchmark, and evaluation protocol for trajectory-level reliability.

References

[1] Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. 2025. MTRAG: A Multi-Turn Conversational Benchmark for Evaluating Retrieval-Augmented Generation Systems. *Transactions of the Association for Computational Linguistics*.

[2] Huaiyu Zhu et al. 2026. MTRAG-UN: A Benchmark for Open Challenges in Multi-Turn Retrieval-Augmented Generation. arXiv preprint.

[3] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045.

[4] Hongru Wang, Wenyu Huang, Yufei Wang, Yuanhao Xi, Jianqiao Lu, Huan Zhang, Nan Hu, Zeming Liu, Jeff Z. Pan, and Kam-Fai Wong. 2025. Rethinking Stateful Tool Use in Multi-Turn Dialogues: Benchmarks and Challenges. *Findings of ACL 2025*.

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv preprint arXiv:2309.15217.

[6] Shi-Qi Yan et al. 2024. Corrective Retrieval Augmented Generation. *International Conference on Learning Representations*.

System	Clean	Over-repaired	Rate
QueryRewrite	44	0	0.0000
SAGE-R	44	0	0.0000
SelfReflect	44	0	0.0000
Vanilla	44	0	0.0000

Table 6: Clean-case over-repair analysis for the deterministic v0 scaffold. The rate is computed over examples whose gold action is answer-as-is.

Comparison	Pairs	Drift	Severity	Repair
A vs key	240	1.0000	1.0000	1.0000
A vs B	48	1.0000	1.0000	1.0000

Table 7: Human-validation workflow scaffold. The current v0 table uses an answer-key proxy and must be replaced by filled human annotations before final reporting.

[7] Yijie Zhu, Haojie Zhou, Wanting Hong, Tian Liu, and Ning Wang. 2026. REAP: Enhancing RAG with Recursive Evaluation and Adaptive Planning for Multi-Hop Question Answering. *Proceedings of the AAAI Conference on Artificial Intelligence*.

[8] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv preprint arXiv:2410.10813.